
CS 301

High-Performance Computing

Lab 06 & 07 - Parallel Interpolation

and Particle Mover

Group No. 25

Yuva Undaviya - 202301426
Manthan Gajera - 202301488

April 19, 2026

Contents

1	Introduction	3
2	Hardware Environment: Lab PC vs. HPC Cluster	3
2.1	Lab PC Architecture	3
2.2	HPC Cluster Architecture	3
3	Assignment 6: Parallel Interpolation (SoA Approach)	4
3.1	Parallel Approach and Race Conditions (Q2)	4
3.2	Pseudocode (Q1)	4
3.3	Illustrative Diagram (Q1)	5
4	Assignment 7: Interpolation with Particle Mover	6
4.1	Parallel Approach and Race Conditions (Q2)	6
4.2	Pseudocode (Q1)	6
4.3	Illustrative Diagram (Q1)	7
5	Results and Performance Analysis	8
5.1	Assignment 6: HPC Cluster Data (Q3)	8
5.2	Assignment 7: Lab PC Benchmarks (Q3)	9
5.3	Assignment 7: HPC Cluster Benchmarks (Q3)	11
5.4	Assignment 7: Phase Analysis (Q11)	12
6	Discussion and Further Analysis	13
6.1	Scalability, Parallel Efficiency, and Atomic Contention (Q4, Q5, Q6)	13
6.2	Phase Analysis: Interpolation vs. Mover (Q11)	13
6.3	Performance Bottlenecks and Load Imbalance (Q8)	13
6.4	Alternative Approaches and Data Structures (Q9, Q10)	13
6.5	Further Optimizations with Unlimited Resources (Q7)	14
7	Conclusion	14

1 Introduction

This report outlines the parallelization of a Particle-in-Cell (PIC) style interpolation algorithm using OpenMP, spanning across two assignments.

- **Assignment 6** focuses purely on the scatter operation (Interpolation), mapping millions of scattered particle values to a structured $N_x \times N_y$ mesh. To optimize this, we employed a Structure of Arrays (SoA) memory layout with fine-grained global atomics.
- **Assignment 7** builds upon this by adding a gather operation (Mover phase), where particles update their physical (x, y) coordinates based on the interpolated mesh forces.

We benchmarked the code against various particle counts and grid sizes across a Lab PC and an HPC Cluster to evaluate performance, parallel efficiency, and scalability.

2 Hardware Environment: Lab PC vs. HPC Cluster

To properly analyze our performance data, it is crucial to understand the architectural differences between our testing systems.

2.1 Lab PC Architecture

The Lab PC generally features a consumer-grade CPU with fewer cores but a higher base clock frequency. It utilizes a unified memory architecture and a single NUMA (Non-Uniform Memory Access) node. For smaller problem configurations, the PC performs exceptionally well due to high single-thread speeds and lower overhead for thread creation. However, as the grid size and particle count increase, it rapidly hits a memory bandwidth bottleneck.

2.2 HPC Cluster Architecture

The HPC Cluster node features enterprise-grade Xeon processors with a much higher core count, larger L3 cache, and a dual-socket NUMA architecture. While individual cores operate at lower clock frequencies compared to the Lab PC, the cluster excels at parallelizing massive workloads across 16+ threads. The dual-socket layout means memory access patterns play a critical role in performance.

3 Assignment 6: Parallel Interpolation (SoA Approach)

3.1 Parallel Approach and Race Conditions (Q2)

The primary challenge in the Interpolation phase is a classic race condition: multiple scattered particles often reside in the same grid cell or adjacent cells, causing concurrent threads to update the exact same grid indices simultaneously.

To resolve this and optimize cache usage simultaneously, we overhauled the data structure from an Array of Structures (AoS) to a **Structure of Arrays (SoA)**. By separating the x and y coordinates into contiguous memory blocks, CPU cache hits improved dramatically. We parallelized the main particle loop using `#pragma omp parallel for schedule(static)`. Instead of using a privatization approach (which consumes excessive memory for large grids), we safely updated the shared global `mesh_value` array using `#pragma omp atomic`.

3.2 Pseudocode (Q1)

```
1 Allocate memory for global mesh_value
2 Allocate Structure of Arrays (SoA) for particles (points.x, points.y)
3 Read binary input into SoA
4
5 For each iteration (0 to Maxiter):
6     1. Interpolation Phase (Scatter):
7         memset(mesh_value, 0)
8
9         #pragma omp parallel for schedule(static)
10        For each particle p (0 to NUM_Points):
11            Read px = points.x[p] and py = points.y[p]
12            Calculate grid indices (i, j)
13            Compute bilinear weights (w00, w10, w01, w11)
14
15            #pragma omp atomic
16            mesh_value[base_idx] += w00
17            #pragma omp atomic
18            mesh_value[base_idx + 1] += w10
19            #pragma omp atomic
20            mesh_value[base_idx + GRID_X] += w01
21            #pragma omp atomic
22            mesh_value[base_idx + GRID_X + 1] += w11
23
24 Save mesh to file and free memory
```

3.3 Illustrative Diagram (Q1)

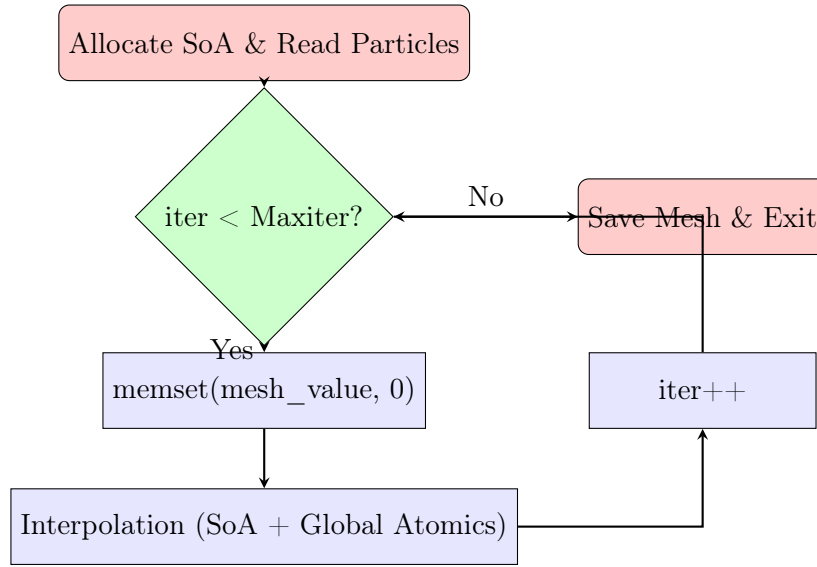


Figure 1: Flowchart of the Assignment 6 Parallel Interpolation Pipeline.

4 Assignment 7: Interpolation with Particle Mover

4.1 Parallel Approach and Race Conditions (Q2)

Assignment 7 extends the pipeline by introducing normalization and a Mover phase. During the Mover phase (Gather: mesh $\rightarrow$ point), particles read from the grid and update their own unique physical coordinates. Since the mesh reads are concurrent and writes are isolated to unique memory locations per thread, **there are no race conditions in the Mover phase**. We simply applied a standard `#pragma omp parallel for`. Boundary checking was also implemented within this loop to mark particles as `is_void` if they exited the domain.

4.2 Pseudocode (Q1)

```
1 Initialize grid and load particles (AoS)
2
3 For each iteration (0 to Maxiter):
4     1. Interpolation Phase (Scatter):
5         memset(mesh_value, 0)
6         #pragma omp parallel for
7         For each particle p:
8             If p is void, continue
9             Calculate grid indices and compute weights
10            #pragma omp atomic
11            Add weights to the four surrounding mesh points
12
13     2. Normalization Phase:
14         #pragma omp parallel reduction(min/max)
15         Find global Min and Max of mesh
16         #pragma omp parallel for
17         Normalize mesh points to range [-1.0, 1.0]
18
19     3. Mover Phase (Gather):
20         #pragma omp parallel for
21         For each particle p:
22             If p is void, continue
23             Gather force F_i using weights from mesh
24             Update position: x += F_i * dx, y += F_i * dy
25             If particle moves outside domain, mark is_void = true
26
27     4. Denormalization Phase:
28         #pragma omp parallel for
29         Restore all grid points to original scale
```

4.3 Illustrative Diagram (Q1)

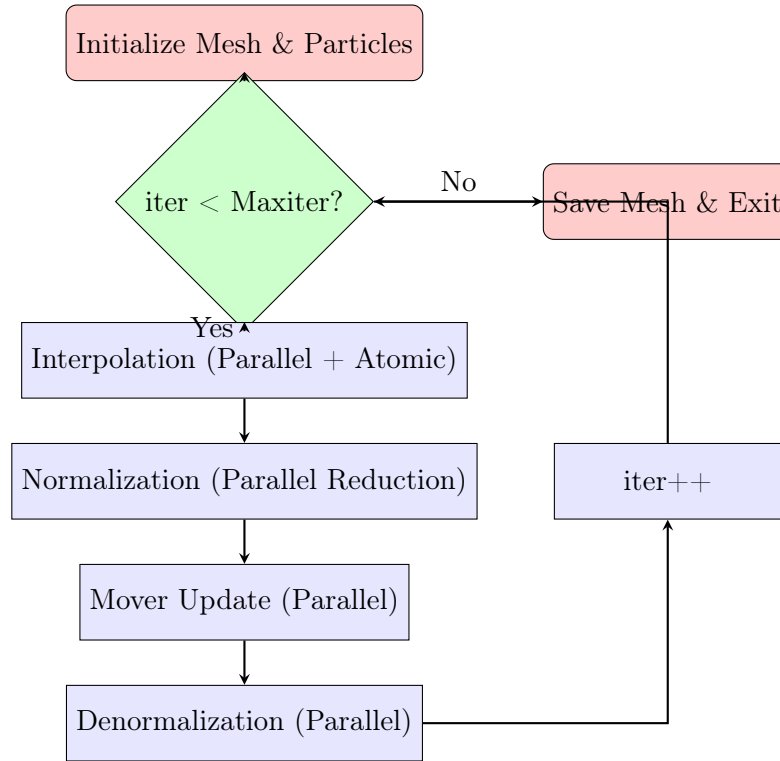


Figure 2: Flowchart of the Assignment 7 Full Pipeline.

5 Results and Performance Analysis

5.1 Assignment 6: HPC Cluster Data (Q3)

In our HPC Cluster experiments for Lab 6, we recorded execution times across different threads for all five configurations. The absolute maximum speedup achieved was **1.79x** on Configuration E (14 million points, 1000×400 grid) utilizing 16 cores.

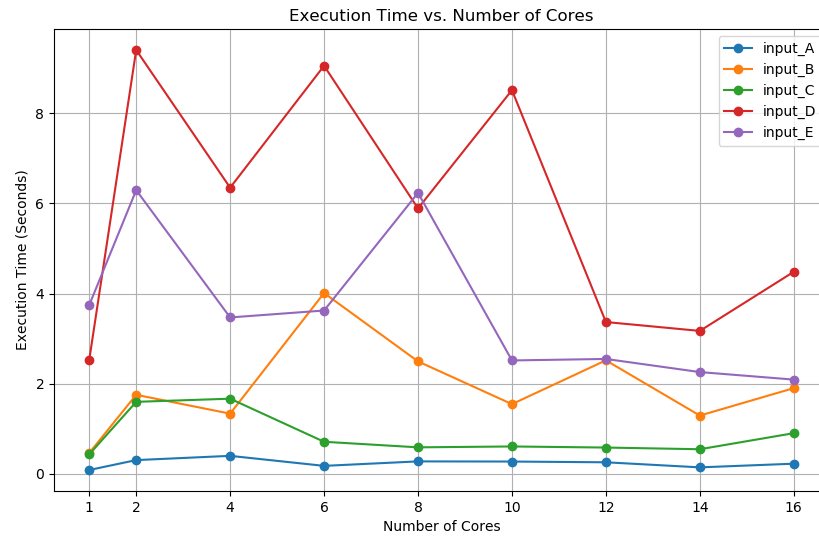


Figure 3: Lab 6 Execution Time vs. Cores on HPC Cluster.

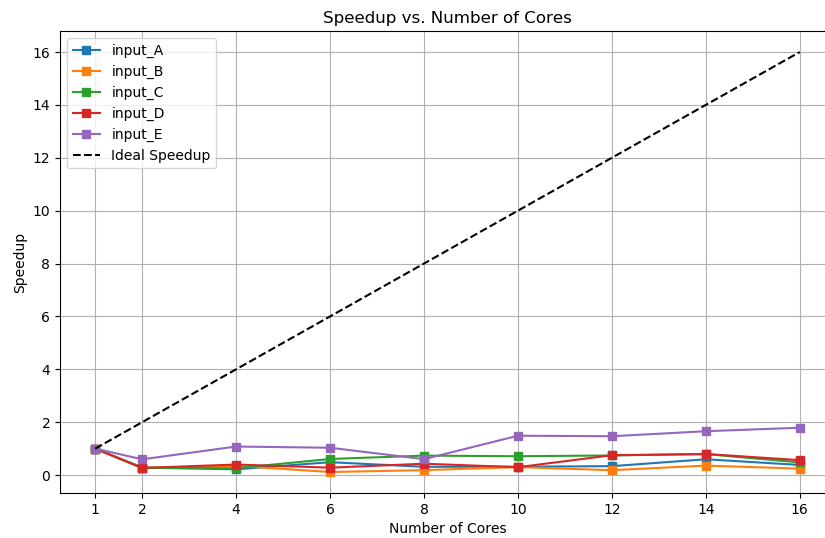


Figure 4: Lab 6 Speedup vs. Cores on HPC Cluster.

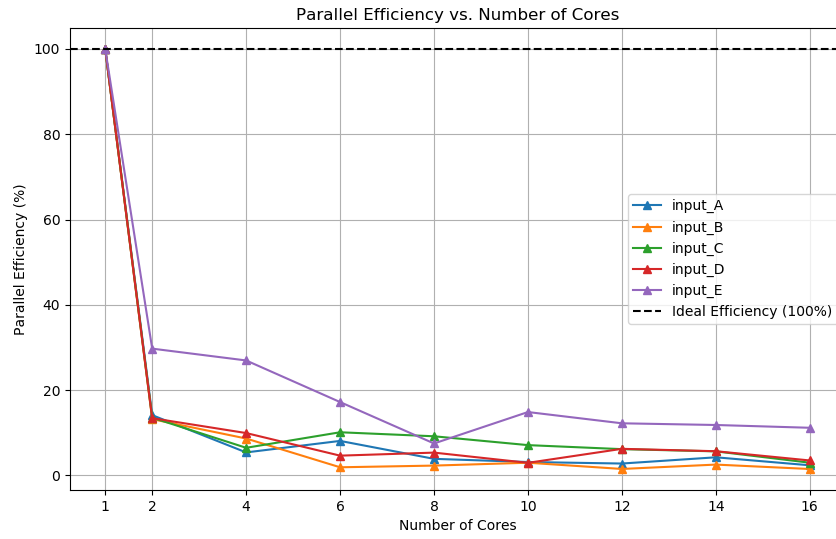


Figure 5: Lab 6 Parallel Efficiency vs. Cores on HPC Cluster.

5.2 Assignment 7: Lab PC Benchmarks (Q3)

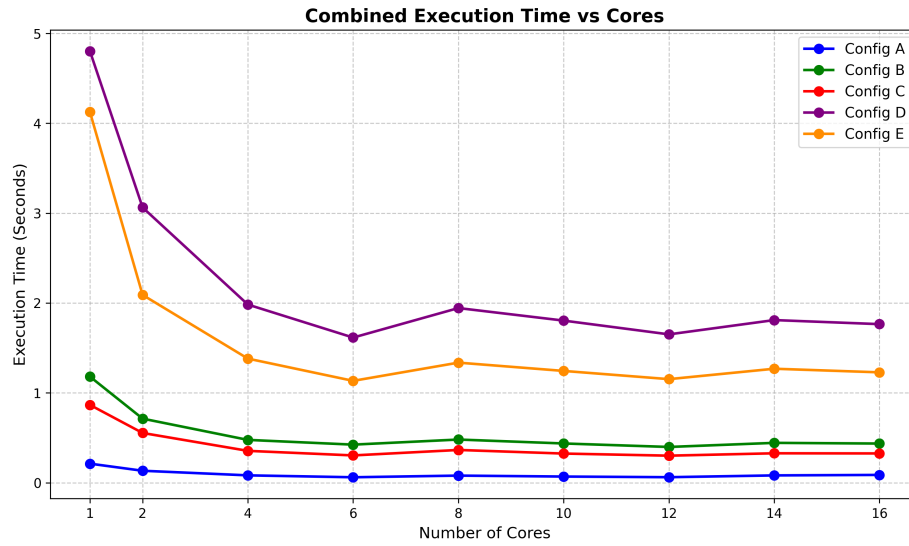


Figure 6: A7 Execution Time vs. Cores for various configurations on Lab PC.

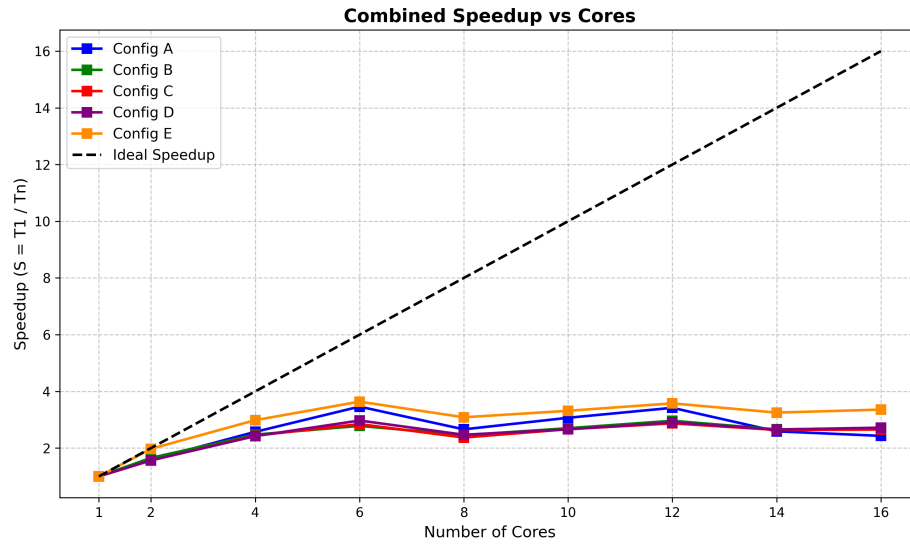


Figure 7: A7 Speedup vs. Cores for various configurations on Lab PC.

5.3 Assignment 7: HPC Cluster Benchmarks (Q3)

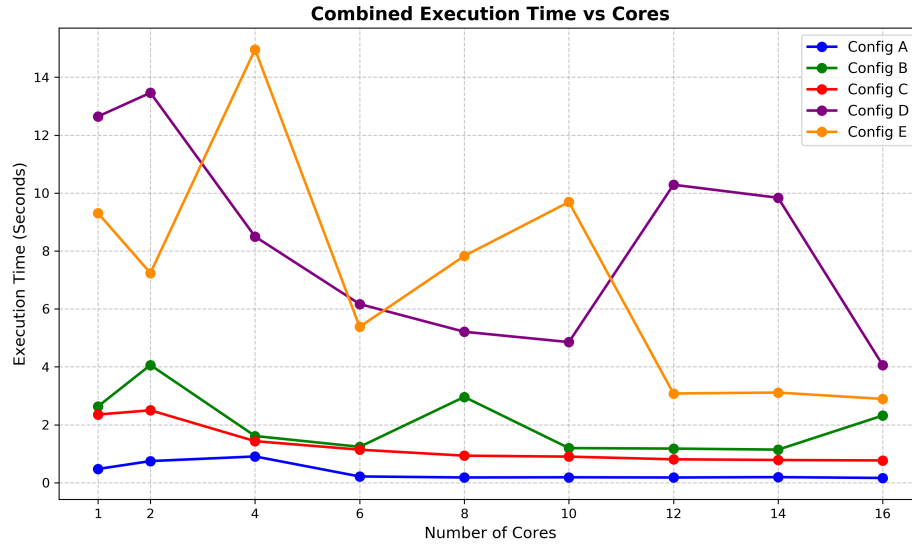


Figure 8: A7 Execution Time vs. Cores for various configurations on HPC Cluster.

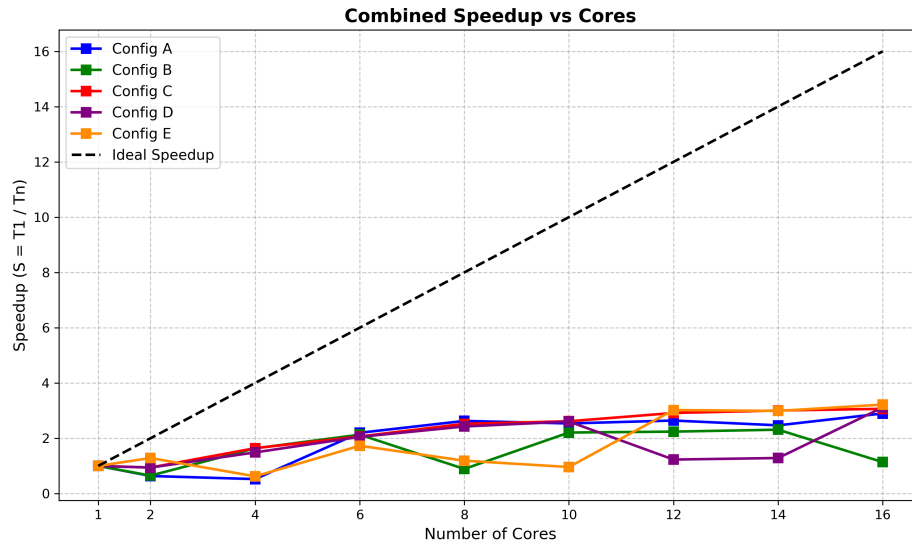


Figure 9: A7 Speedup vs. Cores for various configurations on HPC Cluster.

5.4 Assignment 7: Phase Analysis (Q11)

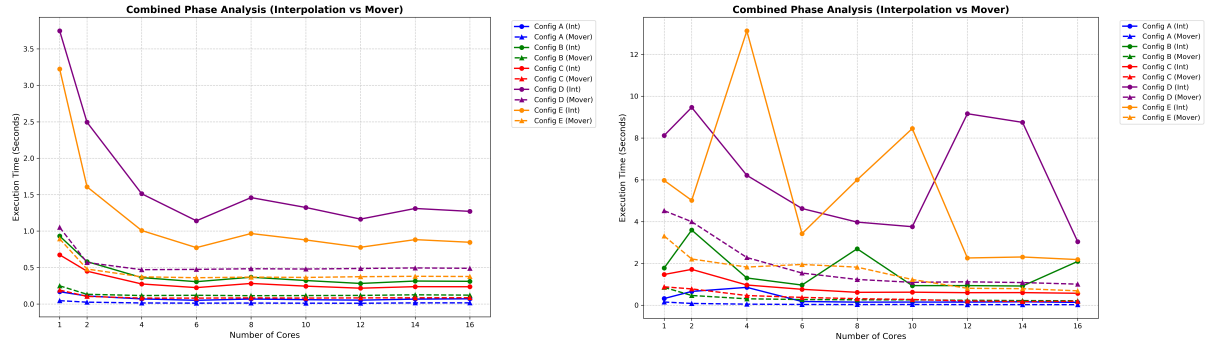


Figure 10: Phase breakdown showing time spent in Interpolation vs Mover (Left: PC, Right: Cluster).

6 Discussion and Further Analysis

6.1 Scalability, Parallel Efficiency, and Atomic Contention (Q4, Q5, Q6)

Upon analyzing the cluster data for Lab 6, a critical observation emerges: for Configurations A, B, C, and D, the parallel implementation performed *slower* than the serial version (speedup $< 1.0x$).

This severe performance penalty is a direct consequence of massive **atomic contention**. Because the grid sizes were relatively small (250×100 and 500×200) compared to the millions of particles, multiple threads frequently attempted to update the exact same grid cells simultaneously. This triggered hardware-level serialization and severe cache coherency traffic (cache line bouncing) across the NUMA nodes, negating the benefits of parallelization.

Only in **Configuration E**, where the grid size was substantially larger (1000×400), did the particle density per cell drop enough to alleviate this atomic contention. With the particles spread out, the parallel computation finally outpaced the serial baseline, resulting in the peak **1.79x** speedup on 16 cores. This highlights that while `#pragma omp atomic` ensures correctness, it scales poorly in high-density shared memory layouts.

6.2 Phase Analysis: Interpolation vs. Mover (Q11)

Analyzing the phase breakdown charts from Lab 7 confirms that the **Interpolation (Scatter) phase is the primary bottleneck**. In this phase, multiple particles map to shared grid indices, forcing the use of atomic operations. Conversely, the Mover phase involves predominantly read operations from the shared mesh and writes to isolated particle structs. Because concurrent reads do not require synchronization, the Mover phase parallelizes highly efficiently and takes up a small fraction of the total execution time.

6.3 Performance Bottlenecks and Load Imbalance (Q8)

Load imbalance manifests in two distinct ways across the assignments:

- **Particle Clustering (Lab 6 & 7):** Particles are not perfectly uniformly distributed. Threads assigned to dense clusters of particles will experience high atomic contention and take much longer to execute than threads processing sparse regions.
- **Void Particles (Lab 7):** Over several iterations, particles move out of the 1×1 domain. Threads assigned chunks of the array containing a high concentration of `is_void` particles will finish their work much faster, leaving CPU cores idling at the implicit OpenMP barrier.

6.4 Alternative Approaches and Data Structures (Q9, Q10)

To improve performance further, we proposed and implemented critical software architectural changes:

- **Structure of Arrays (SoA):** As demonstrated in our Lab 6 implementation, refactoring the code from an Array of Structures (`struct { double x, y; }`) to a Structure of Arrays (`struct { double* x; double* y; }`) drastically improved spatial locality in the CPU cache and minimized memory fetching overhead.

- **Spatial Sorting / Binning:** Before interpolation, particles can be sorted or hashed into "bins" corresponding to their grid cells. This eliminates race conditions entirely, as a thread can safely be assigned an entire bin without overlapping with other threads, effectively removing the need for atomics.

6.5 Further Optimizations with Unlimited Resources (Q7)

If given unlimited HPC resources, we would pivot from a strictly shared-memory OpenMP approach to a Hybrid distributed-memory approach:

1. **MPI for Domain Decomposition:** Split the $M \times M$ grid across multiple compute nodes. Each node would only track and compute particles within its subdomain, utilizing "ghost cells" to handle boundary particle transfers.
2. **GPU Acceleration (CUDA/OpenACC):** The PIC operations (processing millions of independent particles) are perfectly suited for the SIMT (Single Instruction, Multiple Threads) architecture of GPUs.

7 Conclusion

These assignments successfully demonstrated the complexities of parallelizing dependent spatial data. While OpenMP easily parallelizes the Mover phase (gather operations), the Interpolation phase (scatter operations) suffers from atomic contention. By benchmarking across different configurations, we observed firsthand how hardware-level serialization and cache coherency traffic can cause multithreading to scale negatively. We conclude that while CPU multithreading offers gains on sparse grids, solving high-density scatter problems effectively requires advanced techniques like spatial sorting or migrating to distributed/GPU architectures.